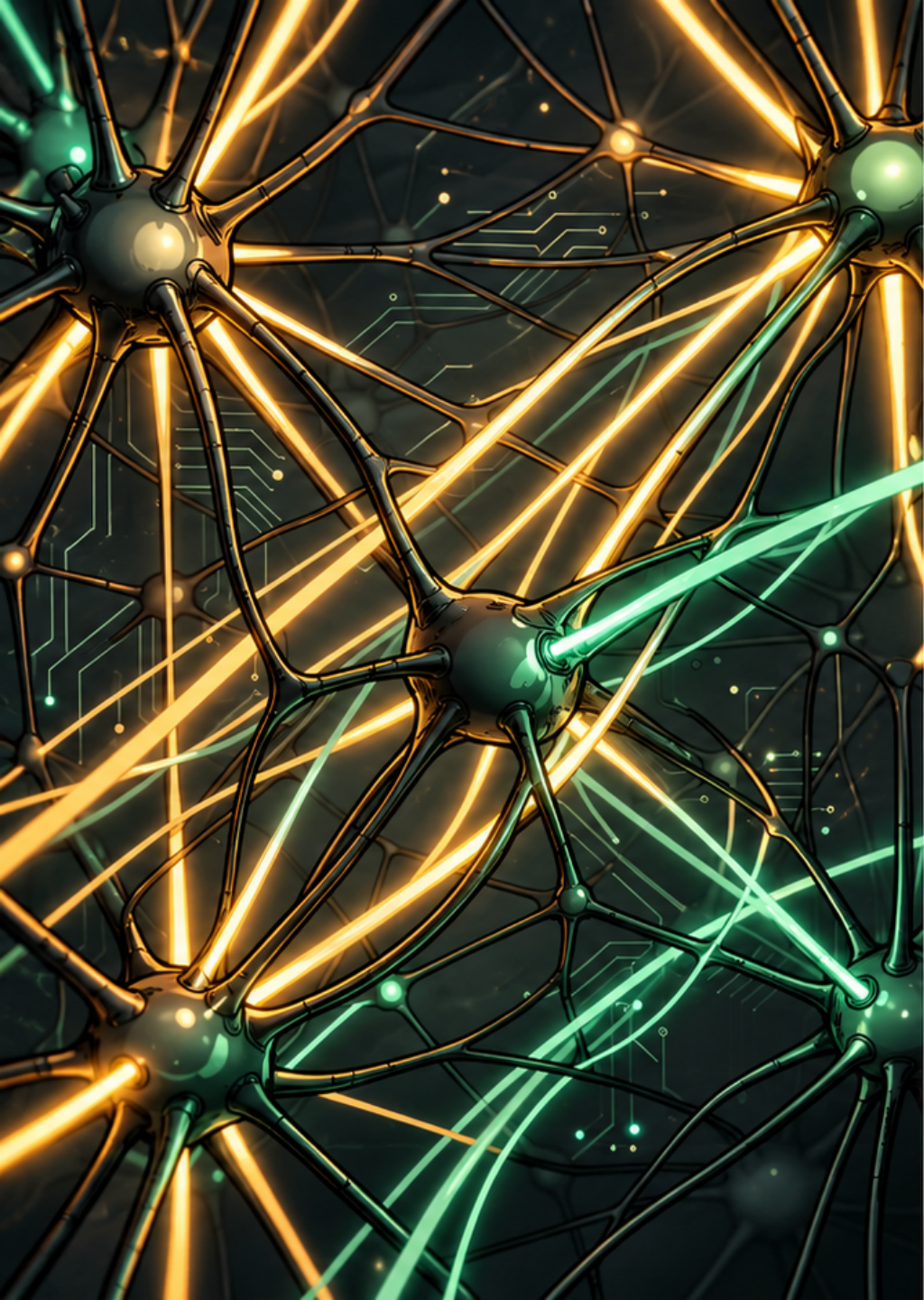




Web Intelligence and NLP-Based Sentiment Analytics

DATA
IDEAS
INSIGHTS
IMPACT

From Web Text to Actionable Intelligence

Shaik Mohammed Kaif

DATA SCIENCE

DATA ANALYTICS

NLP

MACHINE LEARNING

ARTIFICIAL INTELLIGENCE

PYTHON

BUSINESS INTELLIGENCE

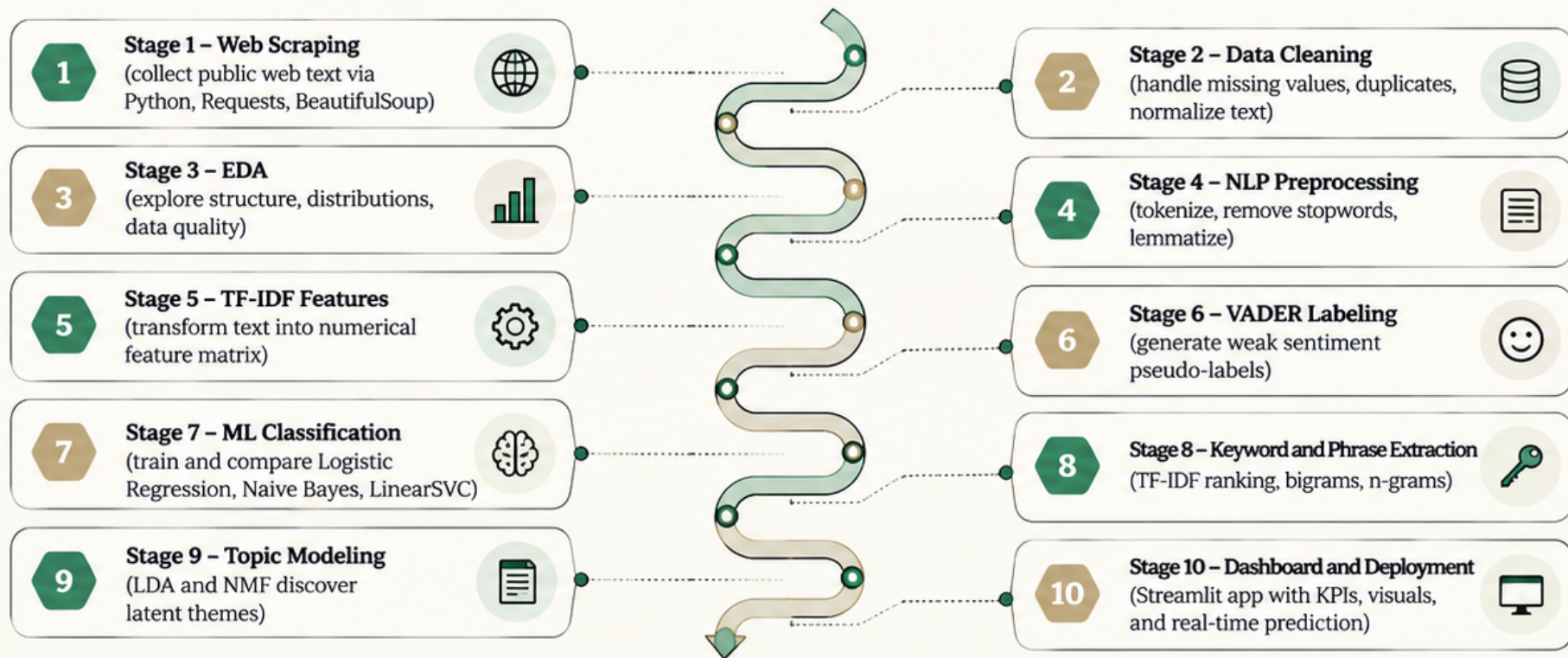


Speaker Notes: This presentation covers an end-to-end NLP analytics system — from collecting raw web text through scraping, to cleaning, preprocessing, sentiment classification, keyword extraction, topic modeling, and finally deploying an interactive Streamlit dashboard with real-time prediction. The central theme is transforming unstructured text into measurable, explainable, and interactive intelligence. The story flows: collect → clean → understand → transform → classify → discover → visualize → predict → deploy. Every step is grounded in practical, reproducible Python tooling.

End-to-End NLP Analytics System

This project is a complete, production-style NLP analytics pipeline that transforms unstructured public web text into a structured, measurable, and interactive intelligence. The system spans the full data science lifecycle: data collection via web scraping, rigorous and exploratory analysis, NLP preprocessing, TF-IDF feature engineering, weak sentiment labeling via VADER, feature engineering for classification, word and phrase extraction, unsupervised topic modeling, and an interactive Streamlit dashboard. Each real-time project. Each stage, designed with reusability. Each stage is modular, reproducible, and deployer notebooks with corresponding source modules.

DATA
TEXT
INSIGHTS
INTELLIGENCE



UNSTRUCTURED
TEXT
TO
STRUCTURED
INSIGHTS

SCRAPE
CLEAN
ANALYZE
MODEL
VISUALIZE
DEPLOY

The pipeline is deliberately modular: each stage produces intermediate artifacts that can be inspected, validated, and reused independently. This design supports iterative development, rigorous quality control, and straightforward extension to new data sources or analytical objectives without reengineering the entire system.



Speaker Notes: This slide establishes the full scope of the project. The pipeline has 10 stages, each building on the previous. Web scraping collects raw HTML and converts it to structured records. Cleaning and EDA prepare and explore the data. NLP preprocessing normalizes text for analysis. TF-IDF converts text to numbers. VADER generates sentiment pseudo-labels because the dataset has no human labels. ML classifiers are trained and compared. Keywords, bigrams, and trigrams reveal important vocabulary. LDA and NMF discover hidden topics. Finally, everything is integrated into a Streamlit dashboard with live prediction. The modular design means each stage can be inspected and improved independently — this is a key design decision worth emphasizing.

The Challenge and What We Set Out to Achieve



The Problem

Web data contains vast quantities of unstructured text that is inherently difficult to analyze at scale. Raw HTML content cannot be consumed directly by analytical systems. Manual interpretation does not scale. Sentiment must be converted into measurable signals. Important keywords and phrase remain hidden without systematic NLP techniques. Topic discovery requires unsupervised methods. And analytical results must be accessible through an interactive, user-friendly interface — not buried in static reports.

“Transform unstructured web text into measurable and interactive intelligence.”



Speaker Notes: The problem is that web text is unstructured and unscalable — you cannot manually read thousands of quotes or reviews. The objectives follow a logical sequence: first collect the data, then clean and explore it, then preprocess the language, then transform it into features, then label it for sentiment, then train classifiers, then extract keywords and discover topics, and finally deploy everything in an interactive dashboard. This is a complete data science workflow, not just a sentiment analysis script. The key message is that every objective connects to the next — this is an integrated system, not a collection of isolated tasks.



Project Objectives

1

Collect — Scrape public web content via Python, Requests, and BeautifulSoup

2

Clean — Prepare analytical-quality data with rigorous validation

3

Explore — Investigate structure, distributions, and data quality

4

Preprocess — Apply NLP normalization, tokenization, and lemmatization

5

Transform — Generate TF-IDF numerical feature representations

6

Label — Generate VADER weak sentiment pseudo-labels

7

Classify — Train and compare multiple supervised ML models

8

Extract — Identify top keywords, bigrams, and trigrams

9

Discover — Uncover latent topics via LDA and NMF

10

Visualize — Build an interactive Streamlit dashboard

11

Predict — Enable real-time sentiment classification on new text

12

Deploy — Package and serve the complete analytical solution

Data Source & Web Scraping Pipeline

Data Source — Quotes to Scrape

The dataset originates from Quotes to Scrape, a publicly available practice web scraping corpus. The source provides structured HTML containing quote text, author names, and associated tags. This is explicitly a practice corpus used to demonstrate an end-to-end web intelligence and NLP workflow — it does not contain real user ratings, purchase data, revenue figures, customer reviews, or human-annotated sentiment labels.

Primary fields extracted: Quote text, Author name, Tags, Source URL.



The dataset is a public practice corpus used to demonstrate an end-to-end web intelligence and NLP workflow. No human sentiment labels are present in the source data.

Scraping Technology Stack



Python — Core programming language



Requests — HTTP client for web requests



BeautifulSoup — HTML parsing and element extraction



Pandas — DataFrame construction and CSV export

Pipeline Flow



Web scraping converts web-based HTML information into structured analytical records, forming the foundation for all downstream NLP and machine learning operations. The output CSV is the single source of truth for all subsequent processing stages.



Speaker Notes: Quotes to Scrape is a well-known practice website for learning web scraping — it is not a commercial dataset. It provides quote text, authors, and tags, but critically, it does NOT have sentiment labels. That's why we need BeautifulSoup to parse and extract the relevant fields, and Pandas to organize everything into a structured CSV. This CSV at `data/raw/scraped_quotes.csv` is the single source of truth for the entire project. Every downstream step depends on this file being clean and consistent.

Data Cleaning, EDA & NLP Preprocessing

FROM RAW TEXT TO CLEAN, ANALYTICAL DATA

CLEAN
EXPLORE
TRANSFORM
ANALYZE
INSIGHTSQUALITY
DATA
BETTER
INSIGHTSTEXT
TO
MEANINGFUL
FEATURES

Data Cleaning Pipeline



Missing-value inspection

Identify and handle null or incomplete records



Duplicate handling

Detect and remove redundant entries



Text normalization

Standardize casing and character encoding



Whitespace cleanup

Remove leading, trailing, and excess internal spaces



Column standardization

Ensure consistent naming and data types



Data type validation

Confirm correct types for analytical operations



Raw Dataset → Clean Dataset → Exploratory Insights

EDA examines dataset structure, author distribution, tag frequency, text length statistics, unique value counts, and overall data quality — establishing a baseline understanding before NLP processing begins.



NLP Text Preprocessing Pipeline

01

Lowercase Conversion

Normalize all text to lowercase for consistency



02

URL & HTML Removal

Strip hyperlinks and markup tags from text



03

Punctuation & Alphabetic Cleanup

Remove symbols and non-letter characters



04

Tokenization

Split text into individual word tokens



05

Stopword Removal

Filter high-frequency, low-information words



06

Lemmatization

Reduce words to dictionary base forms

**Example:** Raw — 'I absolutely LOVE this amazing experience!'

→ Processed — 'absolutely love amazing experience'

Preprocessing reduces noise, normalizes language variation, improves feature quality, and prepares text for reliable machine learning input.



Speaker Notes: This slide covers two critical preparation stages. First, data cleaning handles missing values, duplicates, whitespace, and data types — ensuring the raw CSV becomes a clean analytical dataset. EDA then explores author distributions, tag frequencies, and text length to understand what we are working with. Second, NLP preprocessing transforms raw text into clean, consistent tokens using lowercase conversion, URL and HTML removal, punctuation stripping, tokenization, stopwords removal, and lemmatization. The example shows how a noisy sentence becomes clean tokens. NLTK is the primary library used here.

TF-IDF Feature Engineering & VADER Sentiment Labeling

TRANSFORM TEXT INTO MEANINGFUL FEATURES



TF-IDF Feature Engineering

Machine learning algorithms require numerical representations of text. TF-IDF (Term Frequency–Inverse Document Frequency) transforms documents into a weighted numerical matrix where each cell reflects a term's importance within a document relative to its frequency across the corpus.

1

TF (Term Frequency)

How often a term appears in a given document.

2

IDF (Inverse Document Frequency)

Penalizes terms that appear frequently across all documents.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \frac{N}{\text{DF}(t)}$$



Text to Features Pipeline

Text Document → Vocabulary → TF-IDF Transformation → Numerical Feature Matrix → Machine Learning.

TF-IDF gives higher weight to words that are informative within a document and relatively uncommon across the corpus — making it ideal for both classification and keyword extraction.



VADER Sentiment Labeling

The scraped dataset does not provide human-annotated sentiment labels. VADER (Valence Aware Dictionary and sEntiment Reasoner) is a rule-based sentiment analyzer optimized for social media and informal text, used here to generate weak pseudo sentiment labels for supervised learning.



Compound ≥ 0.05

→ Positive

POSITIVE



Compound ≤ -0.05

→ Negative

NEGATIVE



Otherwise

→ Neutral

NEUTRAL



Methodological Note

VADER-generated sentiment labels are weak/pseudo labels rather than human-annotated ground truth. Model performance metrics reflect agreement with VADER, not human judgment.



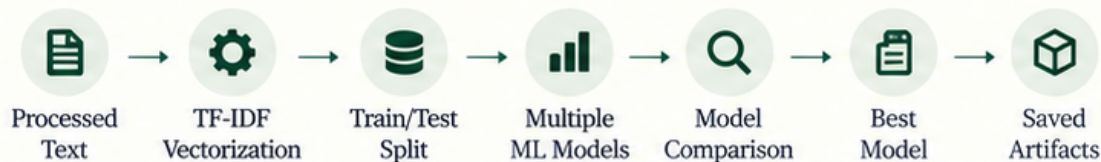
Speaker Notes: This slide covers two foundational steps. First, TF-IDF: ML models need numbers, not text. TF-IDF converts each document into a vector where each dimension is a word weighted by how important it is in that document versus the whole corpus. Second, VADER: because the dataset has no human sentiment labels, we use VADER — a rule-based lexicon tool — to generate pseudo labels. The compound score thresholds are: above 0.05 is positive, below -0.05 is negative, and everything else is neutral. Critically, these are NOT ground truth labels. This is a methodological limitation that must be acknowledged — model metrics measure agreement with VADER, not with human judgment. This distinction is essential for technical accuracy.

Supervised Sentiment Classification & Model Evaluation

BUILDING AND EVALUATING MODELS ON TF-IDF FEATURES



ML Pipeline & Models



Three classifiers are trained and compared to evaluate different baseline approaches and select the strongest performance:



Logistic Regression

Linear probabilistic classifier with L2 regularization; strong baseline for sparse text features.



Multinomial Naive Bayes

Probabilistic classifier based on Bayes' theorem; well-suited for word count and frequency data.



Linear SVC

Support Vector Classifier with linear kernel; effective in high-dimensional sparse text spaces.



Saved artifacts:

`models/text_sentiment_model.pkl` and `models/tfidf_vectorizer.pkl` via joblib for dashboard inference.



Evaluation Metrics



Accuracy

Overall correct classification rate



Weighted Precision

Correctness of positive predictions per class



Weighted Recall

Coverage of actual positives per class



Weighted F1

Harmonic mean of precision and recall



Confusion Matrix

Per-class prediction breakdown



Important Note

Insert actual project metrics. These metrics measure agreement with VADER-generated weak labels, not human-annotated ground truth. LinearSVC decision scores are not calibrated probabilities.



Speaker Notes: Three classifiers are trained on the TF-IDF features using VADER pseudo-labels: Logistic Regression, Multinomial Naive Bayes, and Linear SVC. Each has different strengths — Logistic Regression is a strong linear baseline, Naive Bayes works well with word frequencies, and Linear SVC handles high-dimensional sparse text effectively. The best model is selected based on weighted F1 score and saved using joblib along with the TF-IDF vectorizer. Evaluation uses accuracy, weighted precision, recall, and F1, plus a confusion matrix. Important caveat: all metrics measure agreement with VADER labels, not human ground truth. Also, LinearSVC outputs decision scores, not calibrated probabilities — this is a technical detail interviewers often ask.